

Listing 8-2 The Constructors for the `__Node` and `BinarySearchTree` Classes

```
class BinarySearchTree(object): # A binary search tree class
...

class __Node(object):         # A node in a binary search tree
    def __init__(             # Constructor takes a key that is
        self,                 # used to determine the position
        key,                   # inside the search tree,
        data,                  # the data associated with the key
        left=None,             # and a left and right child node
        right=None):          # if known
        self.key = key         # Copy parameters to instance
        self.data = data       # attributes of the object
        self.leftChild = left
        self.rightChild = right

    def __str__(self):         # Represent a node as a string
        return "{" + str(self.key) + ", " + str(self.data) + "}"

    def __init__(self):        # The tree organizes nodes by their
        self.__root = None    # keys. Initially, it is empty.

    def isEmpty(self):         # Check for empty tree
        return self.__root is None

    def root(self):            # Get the data and key of the root node
        if self.isEmpty():     # If the tree is empty, raise exception
            raise Exception("No root node in empty tree")
        return (               # Otherwise return root data and its key
            self.__root.data, self.__root.key)
```

The `__Node` objects are created and manipulated by the `BinarySearchTree`'s methods. The fields inside `__Node` can be declared as public attributes because the class is private within the `BinarySearchTree`. This declaration allows for direct reading and writing without making accessor methods like `getKey()` or `setData()`. The `__Node` constructor simply populates the fields from the arguments provided. If the child nodes are not provided, the fields for their references are filled with `None`.

We add a `__str__()` method for `__Node` objects to aid in displaying the contents while debugging. Notably, it does not show the child nodes. We

discuss how to display full trees a little later. That's all the methods needed for `__Node` objects; all the rest of the methods you define are for `BinarySearchTree` objects.

[Listing 8-2](#) shows an `isEmpty()` method for `BinarySearchTree` objects that checks whether the tree has any nodes in it. The `root()` method extracts the root node's data and key. It's like `peek()` for a queue and throws an exception if the tree is empty.

Some programmers also include a reference to a node's parent in the `__Node` class. Doing so simplifies some operations but complicates others, so we don't include it here. Adding a parent reference achieves something similar to the `DoublyLinkedList` class described in [Chapter 5](#), “[Linked Lists](#)”; it's useful in certain contexts but adds complexity.

We've made another design choice by storing the key for each node in its own field. For the data structures based on arrays, we chose to use a key function that extracts the key from each array item. That approach was more convenient for arrays because storing the keys separately from the data would require the equivalent of a key array along with the data array. In the case of node class with named fields, adding a key field makes the code perhaps more readable and somewhat more efficient by avoiding some function calls. It also makes the key more independent of the data, which adds flexibility and can be used to enforce constraints like immutable keys even when data changes. The `BinarySearchTree` class has several methods. They are used for finding, inserting, deleting, and traversing nodes; and for displaying the tree. We investigate them each separately.

Finding a Node

Finding a node with a specific key is the simplest of the major tree operations. It's also the most important because it is essential to the binary search tree's purpose.

The visualization tool shows only the key for each node and a color for its data. Keep in mind that the purpose of the data structure is to store a collection of records, not just the key or a simple color. The keys can be more than simple integers; any data type that can be ordered could be used. The visualization and examples shown here use integers for brevity. After a node is discovered by its key, it's the data that's returned to the caller, not the node itself.

Using the Visualization Tool to Find a Node

Look at the visualization tool and pick a node, preferably one near the bottom of the tree (as far from the root as possible). The number shown in this node is its *key value*. We're going to demonstrate how the visualization tool finds the node, given the key value.

For purposes of this discussion, we choose to find the node holding the item with key value 50, as shown in [Figure 8-8](#). Of course, when you run the visualization tool, you may get a different tree and may need to pick a different key value.

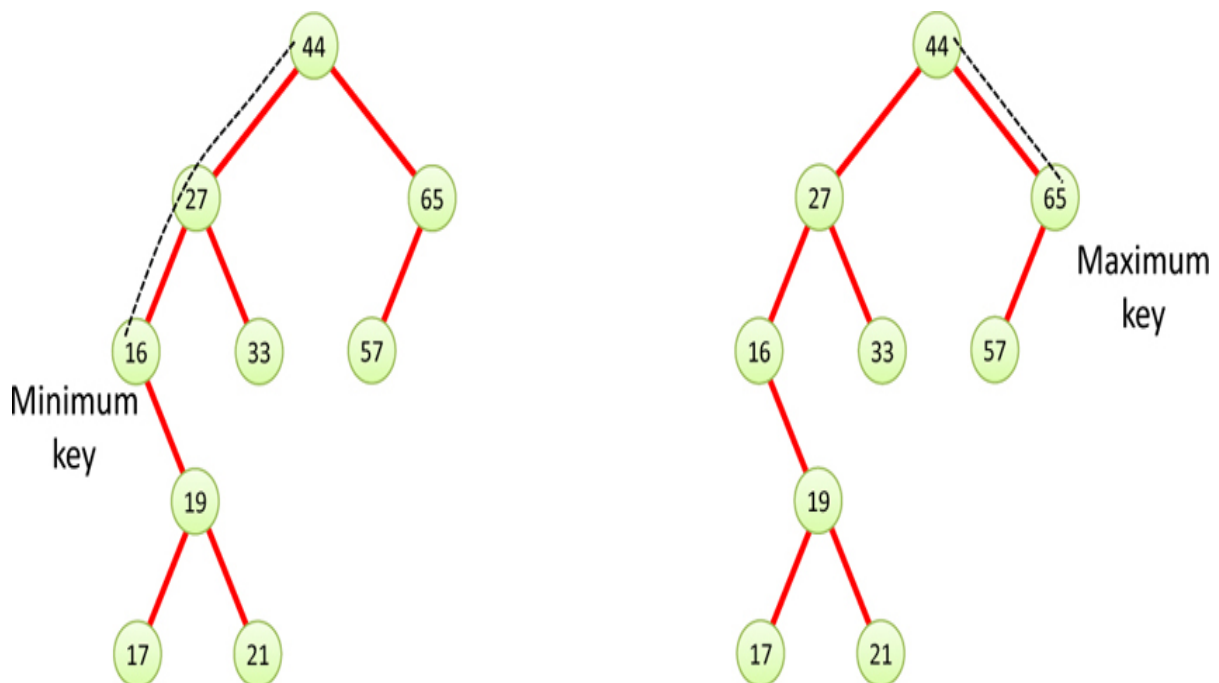


Figure 8-8 *Finding the node with key 50*

Enter the key value in the text entry box, hold down the Shift key, and select the Search button, and then the Step button, ►. By repeatedly pressing the Step button, you can see all the individual steps taken to find key 50. On the second press, the `current` pointer shows up at the root of the tree, as seen in [Figure 8-8](#). On the next click, a `parent` pointer shows up that will follow the `current` pointer. Ignore that pointer and the code display for a moment; we describe them in detail shortly.

As the visualization tool looks for the specified node, it makes a decision at the `current` node. It compares the desired key with the one found at the current

node. If it's the same, it's found the desired node and can quit. If not, it must decide where to look next.

In [Figure 8-8](#) the `current` arrow starts at the root. The program compares the goal key value 50 with the value at the root, which is 77. The goal key is less, so the program knows the desired node must be on the left side of the tree—either the root's left child or one of this child's descendants. The left child of the root has the value 59, so the comparison of 50 and 59 will show that the desired node is in the left subtree of 59. The `current` arrow goes to 46, the root of this subtree. This time, 50 is greater than the 46 node, so it goes to the right, to node 56, as shown in [Figure 8-9](#). A few steps later, comparing 50 with 56 leads the program to the left child. The comparison at that leaf node shows that 50 equals the node's key value, so it has found the node we sought.



Figure 8-9 *The second to last step in finding key 50*

The visualization tool changes a little after it finds the desired node. The `current` arrow changes into the `node` arrow (and `parent` changes into `p`). That's because of the way variables are named in the code, which we show in the next section. The tool doesn't do anything with the node after finding it, except to encircle it and display a message saying it has been found. A serious program would perform some operation on the found node, such as displaying its contents or changing one of its fields.

Python Code for Finding a Node

[Listing 8-3](#) shows the code for the `__find()` and `search()` methods. The `__find()` method is private because it can return a node object. Callers of the

BinarySearchTree class use the `search()` method to get the data stored in a node.

Listing 8-3 *The Methods to Find a Binary Search Tree Node Based on Its Key*

```
class BinarySearchTree(object): # A binary search tree class
...
    def __find(self, goal):      # Find an internal node whose key
        current = self.__root    # matches goal and its parent. Start at
        parent = self            # root and track parent of current node
        while (current and      # While there is a tree left to explore
                goal != current.key): # and current key isn't the goal
            parent = current      # Prepare to move one level down
            current = (           # Advance current to left subtree when
                current.leftChild if goal < current.key else # goal is
                current.rightChild) # less than current key, else right

        # If the loop ended on a node, it must have the goal key
        return (current, parent) # Return the node or None and parent

    def search(self, goal):      # Public method to get data associated
        node, p = self.__find(goal) # with a goal key. First, find node
        return node.data if node else None # w/ goal & return any data
```

The only argument to `__find()` is `goal`, the key value to be found. This routine creates the variable `current` to hold the node currently being examined. The routine starts at the root – the only node it can access directly. That is, it sets `current` to the root. It also sets a `parent` variable to `self`, which is the tree object. In the visualization tool, `parent` starts off pointing at the tree object. Because parent links are not stored in the nodes, the `__find()` method tracks the parent node of `current` so that it can return it to the caller along with the goal node. This capability will be very useful in other methods. The `parent` variable is always either the `BinarySearchTree` being searched or one of its `__Node` objects.

In the `while` loop, `__find()` first confirms that `current` is not `None` and references some existing node. If it doesn't, the search has gone beyond a leaf node (or started with an empty tree), and the goal node isn't in the tree. The second part of the `while` test compares the value to be found, `goal`, with the value of the `current` node's `key` field. If the key matches, then the loop is done. If it doesn't, then `current` needs to advance to the appropriate subtree.

First, it updates `parent` to be the `current` node and then updates `current`. If `goal` is less than `current`'s key, `current` advances to its left child. If `goal` is greater than `current`'s key, `current` advances to its right child.

Can't Find the Node

If `current` becomes equal to `None`, you've reached the end of the line without finding the node you were looking for, so it can't be in the tree. That could happen if the root node was `None` or if following the child links led to a node without a child (on the side where the goal key would go). Both the `current` node (`None`) and its parent are returned to the caller to indicate the result. In the visualization tool, try entering a key that doesn't appear in the tree and select Search. You then see the `current` pointer descend through the existing nodes and land on a spot where the key should be found but no node exists. Pointing to "empty space" indicates that the variable's value is `None`.

Found the Node

If the condition of the `while` loop is not satisfied while `current` references some node in the tree, then the loop exits, and the `current` key must be the goal. That is, it has found the node being sought and `current` references it. It returns the node reference along with the parent reference so that the routine that called `__find()` can access any of the node's (or its parent's) data. Note that it returns the value of `current` for both success and failure of finding the key; it is `None` when the goal isn't found.

The `search()` method calls the `__find()` method to set its `node` and `parent (p)` variables. That's what you see in the visualization tool after the `__find()` method returns. If a non-`None` reference was found, `search()` returns the data for that node. In this case, the method assumes that data items stored in the nodes can never be `None`; otherwise, the caller would not be able to distinguish them.

Tree Efficiency

As you can see, the time required to find a node depends on its depth in the tree, the number of levels below the root. If the tree is balanced, this is $O(\log N)$ time, or more specifically $O(\log_2 N)$ time, the logarithm to base 2, where N is the number of nodes. It's just like the binary search done in arrays where half

the nodes were eliminated after each comparison. A fully balanced tree is the best case. In the worst case, the tree is completely unbalanced, like the examples shown in [Figure 8-6](#), and the time required is $O(N)$. We discuss the efficiency of `__find()` and other operations toward the end of this chapter.

Inserting a Node

To insert a node, you must first find the place to insert it. This is the same process as trying to find a node that turns out not to exist, as described in the earlier “Can’t Find the Node” section. You follow the path from the root toward the appropriate node. This is either a node with the same key as the node to be inserted or `None`, if this is a new key. If it’s the same key, you could try to insert it in the right subtree, but doing so adds some complexity. Another option is to replace the data for that node with the new data. For now, we allow only unique keys to be inserted; we discuss duplicate keys later.

If the key to insert is not in the tree, then `__find()` returns `None` for the reference to the node along with a parent reference. The new node is connected as the parent’s left or right child, depending on whether the new node’s key is less or greater than that of the parent. If the parent reference returned by `__find()` is `self`, the `BinarySearchTree` itself, then the node becomes the root node.

[Figure 8-10](#) illustrates the process of inserting a node, with key 31, into a tree. The `__find(31)` method starts walking the path from the root node. Because 31 is less than the root node key, 44, it follows the left child link. That child’s key is 27, so it follows that child’s right child link. There it encounters key 33, so it again follows the left child link. That is `None`, so `__find(31)` stops with the parent pointing at the leaf node with key 33. The new leaf node with key 31 is created, and the parent’s left child link is set to reference it.

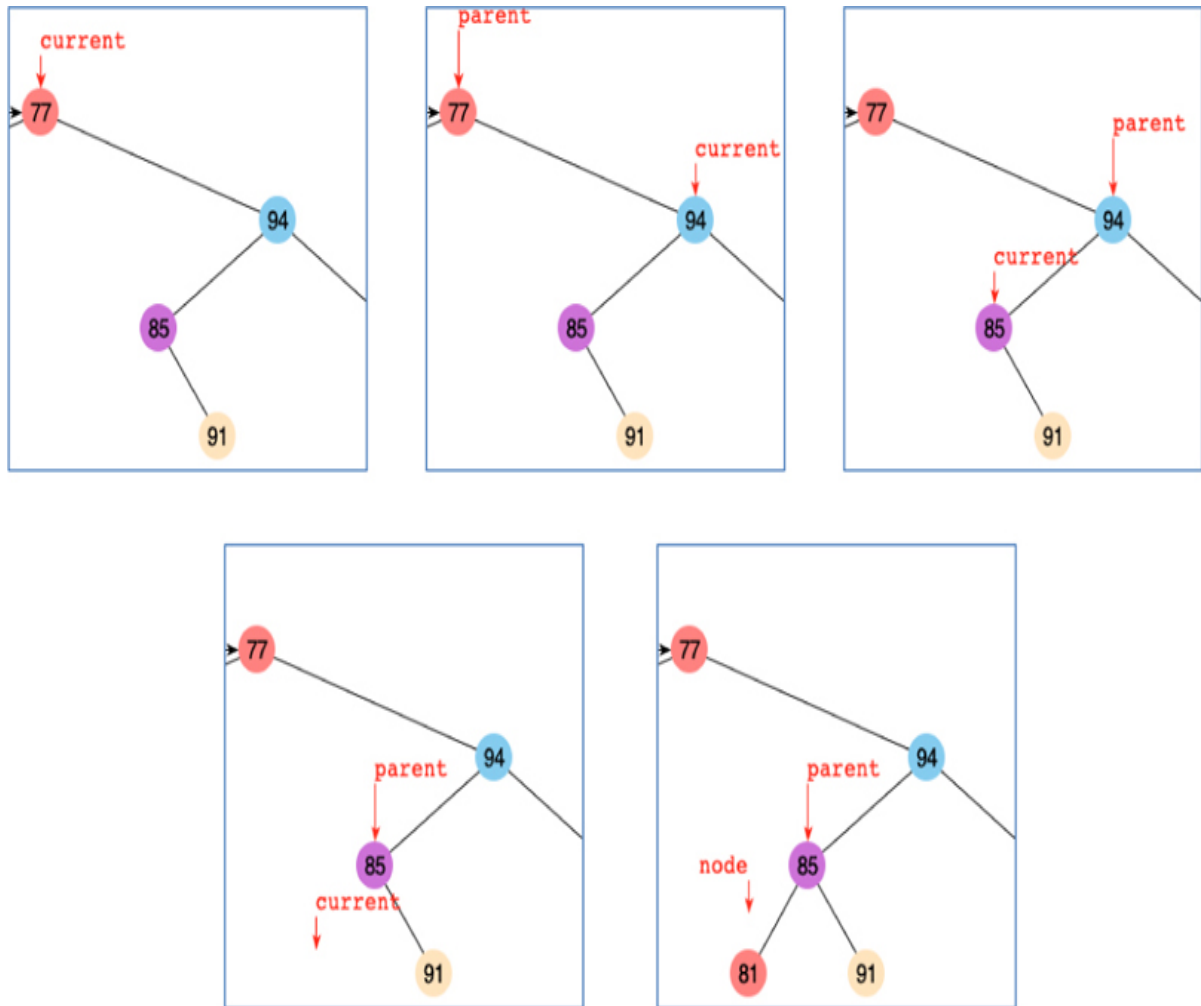


Figure 8-10 *Inserting a node in binary search tree*

Using the Visualization Tool to Insert a Node

To insert a new node with the visualization tool, enter a key value that's not in the tree and select the Insert button. The first step for the program is to find where it should be inserted. For example, inserting 81 into the tree from an earlier example calls the `__find()` method of [Listing 8-3](#), which causes the search to follow the path shown in [Figure 8-11](#).



Figure 8-11 *Steps for inserting a node with key 81 using the visualization tool*

The `current` pointer starts at the root node with key 77. Finding 81 to be larger, it goes to the right child, node 94. Now the key to insert is less than the current key, so it descends to node 85. The `parent` pointer follows the `current` pointer at each of these steps. When `current` reaches node 85, it goes to its left child and finds it missing. The call to `__find()` returns `None` and the `parent` pointer.

After locating the `parent` node with the empty child where the new key should go, the visualization tool creates a new node with the key 81, some data represented by a color, and sets the left child pointer of node 85, the parent, to point at it. The `node` pointer returned by `__find()` is moved away because it still is `None`.

Python Code for Inserting a Node

The `insert()` method takes parameters for the key and data to insert, as shown in [Listing 8-4](#). It calls the `__find()` method with the new node's key to determine whether that key already exists and where its parent node should be. This implementation allows only unique keys in the tree, so if it finds a node with the same key, `insert()` updates the data for that key and returns `False` to indicate that no new node was created.

Listing 8-4 *The `insert()` Method of `BinarySearchTree`*

```

class BinarySearchTree(object): # A binary search tree class
...
    def insert(self,          # Insert a new node in a binary
                key,          # search tree finding where its key
                data):        # places it and storing its data
    node, parent = self.__find( # Try finding the key in the tree
                                key) # and getting its parent node
    if node:                    # If we find a node with this key,
        node.data = data       # then update the node's data
        return False           # and return flag for no insertion

    if parent is self:         # For empty trees, insert new node at
        self.__root = self.__Node(key, data) # root of tree
    elif key < parent.key:     # If new key is less than parent's key,
        parent.leftChild = self.__Node( # insert new node as left
            key, data, right=node) # child of parent
    else:                      # Otherwise insert new node as right
        parent.rightChild = self.__Node( # child of parent
            key, data, right=node)
    return True                # Return flag for valid insertion

```

If a matching node was not found, then insertion depends on what kind of parent reference was returned from `__find()`. If it's `self`, the `BinarySearchTree` must be empty, so the new node becomes the root node of the tree. Otherwise, the parent is a node, so `insert()` decides which child will get the new node by comparing the new node's `key` with that of the parent. If the new key is lower, then the new node becomes the left child; otherwise, it becomes the right child. Finally, `insert()` returns `True` to indicate the insertion succeeded.

When `insert()` creates the new node, it sets the new node's right child to the node returned from `__find()`. You might wonder why that's there, especially because `node` can only be `None` at that point (if it were not `None`, `insert()` would have returned `False` before reaching that point). The reason goes back to what to do with duplicate keys. If you allow nodes with duplicate keys, then you must put them somewhere. The binary search tree definition says that a node's key is less than or equal to that of its right child. So, if you allow duplicate keys, the duplicate node cannot go in the left child. By specifying something other than `None` as the right child of the new node, other nodes with the same key can be retained. We leave as an exercise how to insert (and delete) nodes with duplicate keys.

Traversing the Tree

Traversing a tree means visiting each node in a specified order. Traversing a tree is useful in some circumstances such as going through all the records to look for ones that need some action (for example, parts of a vehicle that are sourced from a particular country). This process may not be as commonly used as finding, inserting, and deleting nodes but it is important nevertheless.

You can traverse a tree in three simple ways. They're called **pre-order**, **in-order**, and **post-order**. The most commonly used order for binary search trees is in-order, so let's look at that first and then return briefly to the other two.

In-order Traversal

An in-order traversal of a binary search tree causes all the nodes to be visited in *ascending order* of their key values. If you want to create a list of the data in a binary tree sorted by their keys, this is one way to do it.

The simplest way to carry out a traversal is the use of recursion (discussed in [Chapter 6](#)). A recursive method to traverse the entire tree is called with a node as an argument. Initially, this node is the root. The method needs to do only three things:

1. Call itself to traverse the node's left subtree.
2. Visit the node.
3. Call itself to traverse the node's right subtree.

Remember that *visiting* a node means doing something to it: displaying it, updating a field, adding it to a queue, writing it to a file, or whatever.

The three traversal orders work with any binary tree, not just with binary search trees. The traversal mechanism doesn't pay any attention to the key values of the nodes; it only concerns itself with the node's children and data. In other words, in-order traversal means "in order of increasing key values" only when the binary search tree criteria are used to place the nodes in the tree. The *in* of *in-order* refers to a node being visited in between the left and right subtrees. The *pre* of *pre-order* means visiting the node before visiting its children, and *post-order* visits the node after visiting the children. This distinction is like the differences between *infix* and *postfix* notation for arithmetic expressions described in [Chapter 4](#), "[Stacks and Queues](#)."

To see how this recursive traversal works, [Figure 8-12](#) shows the calls that happen during an in-order traversal of a small binary tree. The `tree` variable references a four-node binary search tree. The figure shows the invocation of an `inOrderTraverse()` method on the tree that will call the `print` function on each of its nodes.

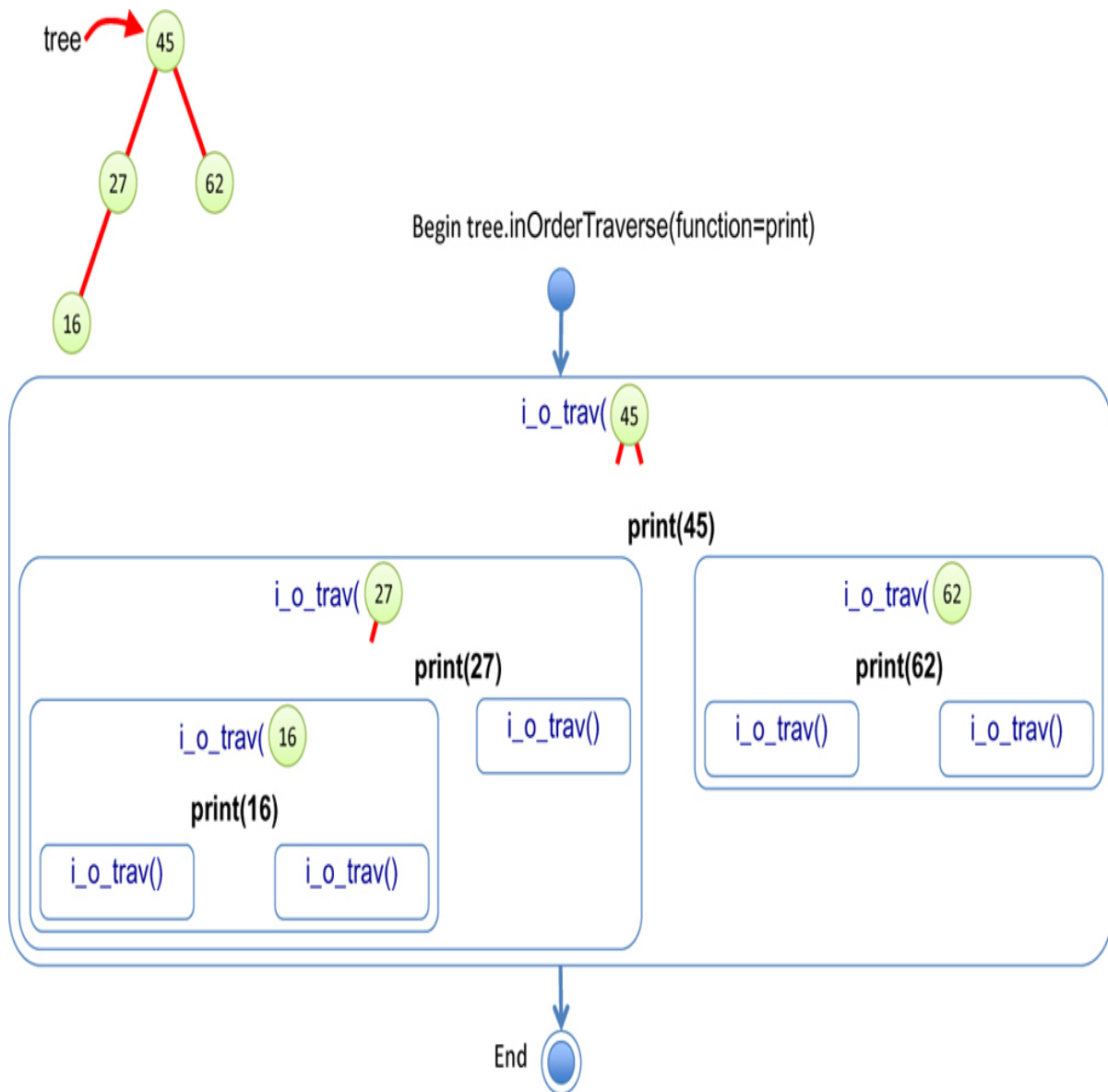


Figure 8-12 *In-order traversal of a small tree*

The blue rounded rectangles show the recursive calls on each subtree. The name of the recursive method has been abbreviated as `i_o_trav()` to fit all the calls in the figure. The first (outermost) call is on the root node (key 45). Each recursive call performs the three steps outlined previously. First, it makes a

recursive call on the left subtree, rooted at key 27. That shows up as another blue rounded rectangle on the left of the figure.

Processing the subtree rooted at key 27 starts by making a recursive call on its left subtree, rooted at key 16. Another rectangle shows that call in the lower left. As before, its first task is to make a recursive call on its left subtree. That subtree is empty because it is a leaf node and is shown in the figure as a call to `i_o_trav()` with no arguments. Because the subtree is empty, nothing happens and the recursive call returns.

Back in the call to `i_o_trav(16)`, it now reaches step 2 and “visits” the node by executing the function, `print`, on the node itself. This is shown in the figure as `print(16)` in black. In general, visiting a node would do more than just print the node’s key; it would take some action on the data stored at the node. The figure doesn’t show that action, but it would occur when the `print(16)` is executed.

After visiting the node with key 16, it’s time for step 3: call itself on the right subtree. The node with key 16 has no right child, which shows up as the smallest-sized rectangle because it is a call on an empty subtree. That completes the execution for the subtree rooted at key 16. Control passes back to the caller, the call on the subtree rooted at key 27.

The rest of the processing progresses similarly. The visit to the node with key 27 executes `print(27)` and then makes a call on its empty right subtree. That finishes the call on node 27 and control passes back to the call on the root of the tree, node 45. After executing `print(45)`, it makes a call to traverse its right (nonempty) subtree. This is the fourth and final node in the tree, node 62. It makes a call on its empty left subtree, executes `print(62)`, and finishes with a call on its empty right subtree. Control passes back up through the call on the root node, 45, and that ends the full tree traversal.

Pre-order and Post-order Traversals

The other two traversal orders are similar: only the timing of visiting the node changes. For pre-order traversal, the node is visited first, and for post-order, it’s visited last. The two subtrees are always visited in the same order: left and then right. [Figure 8-13](#) shows the execution of a pre-order traversal on the same four-node tree as in [Figure 8-12](#). The execution of the `print()` function happens before visiting the two subtrees. That means that the pre-order

traversal would print 45, 27, 16, 62 compared to the in-order traversal's 16, 27, 45, 62. As the figures show, the differences between the orders are small, but the overall effect is large.

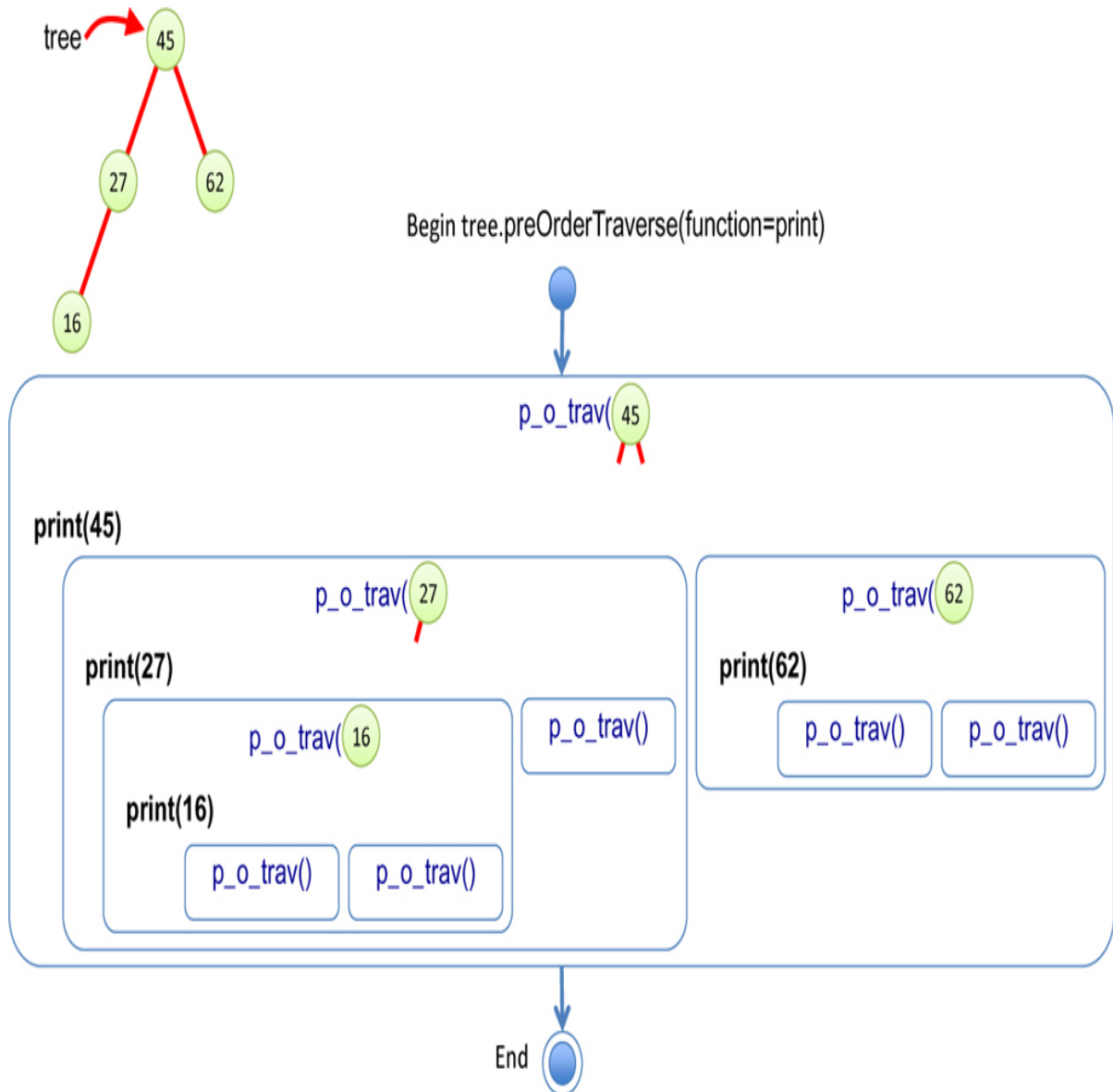


Figure 8-13 *Pre-order traversal of a small tree*

Python Code for Traversing

Let's look at simple way of implementing the in-order traversal now. As you saw in stacks, queues, linked lists, and other data structures, it's straightforward to define the traversal using a function passed as an argument that gets applied

to each item stored in the structure. The interesting difference with trees is that recursion makes it very compact.

Because these trees are represented using two classes, `BinarySearchTree` and `__Node`, you need methods that can operate on both types of object. In [Listing 8-5](#), the `inOrderTraverse()` method handles the traversal on `BinarySearchTree` objects. It serves as the public interface to the traversal and calls a private method `__inOrderTraverse()` to do the actual work on subtrees. It passes the root node to the private method and returns.

Listing 8-5 *Recursive Implementation of `inOrderTraverse()`*

```
class BinarySearchTree(object):    # A binary search tree class
...
    def inOrderTraverse(          # Visit all nodes of the tree in-order
        self, function=print):   # and apply a function to each node
        self.__inOrderTraverse(  # Call recursive version starting at
            self.__root, function=function) # root node

    def __inOrderTraverse(        # Visit a subtree in-order, recursively
        self, node, function):   # The subtree's root is node
        if node:                 # Check that this is real subtree
            self.__inOrderTraverse( # Traverse the left subtree
                node.leftChild, function)
            function(node)         # Visit this node
            self.__inOrderTraverse( # Traverse the right subtree
                node.rightChild, function)
```

The private method expects a `__Node` object (or `None`) for its `node` parameter and performs the three steps on the subtree rooted at the node. First, it checks `node` and returns immediately if it is `None` because that signifies an empty subtree. For legitimate nodes, it first makes a recursive call to itself to process the left child of the node. Second, it visits the `node` by invoking the `function` on it. Third, it makes a recursive call to process the node's right child. That's all there is to it.

Using a Generator for Traversal

The `inOrderTraverse()` method is straightforward, but it has at least three shortcomings. First, to implement the other orderings, you would either need to write more methods or add a parameter that specifies the ordering to perform.

Second, the `function` passed as an argument to “visit” each node needs to take a `__Node` object as argument. That’s a private class inside the `BinarySearchTree` that protects the nodes from being manipulated by the caller. One alternative that avoids passing a reference to a `__Node` object would be to pass in only the data field (and maybe the key field) of each node as arguments to the `visit function`. That approach would minimize what the caller could do to the node and prevent it from altering the other node references.

Third, using a function to describe the action to perform on each node has its limitations. Typically functions perform the same operation each time they are invoked and don’t know about the history of previous calls. During the traversal of a data structure like a tree, being able to make use of the results of previous node visits dramatically expands the possible operations. Here are some examples that you might want to do:

- Add up all the values in a particular field at every node.
- Get a list of all the unique strings in a field from every node.
- Add the node’s key to some list if none of the previously visited nodes have a bigger value in some field.

In all these traversals, it’s very convenient to be able to accumulate results somewhere during the traversal. That’s possible to do with functions, but generators make it easier. We introduced generators in [Chapter 5](#), and because trees share many similarities with those structures, they are very useful for traversing trees.

We address these shortcomings in a recursive generator version of the `traverse` method, `traverse_rec()`, shown in [Listing 8-6](#). This version adds some complexity to the code but makes using traversal much easier. First, we add a parameter, `traverseType`, to the `traverse_rec()` method so that we don’t need three separate `traverse` routines. The first `if` statement verifies that this parameter is one of the three supported orderings: `pre`, `in`, and `post`. If not, it raises an exception. Otherwise, it launches the recursive private method, `__traverse()`, starting with the root node, just like `inOrderTraverse()` does.

Listing 8-6 *The Recursive Generator for Traversal*

```
class BinarySearchTree(object):    # A binary search tree class
```



```

...
def traverse_rec(self,          # Traverse the tree recursively in
                  traverseType="in"): # pre, in, or post order
    if traverseType in [      # Verify type is an accepted value and
        'pre', 'in', 'post']: # use generator to walk the tree
        return self.__traverse( # yielding (key, data) pairs
                                self.__root, traverseType) # starting at root

    raise ValueError("Unknown traversal type: " + str(traverseType))

def __traverse(self,          # Recursive generator to traverse
               node,          # subtree rooted at node in pre, in, or
               traverseType): # post order
    if node is None:          # If subtree is empty,
        return                # traversal is done
    if traverseType == "pre": # For pre-order, yield the current
        yield (node.key, node.data) # node before all the others
    for childKey, childData in self.__traverse( # Recursively
        node.leftChild, traverseType): # traverse the left subtree
        yield (childKey, childData) # yielding its nodes
    if traverseType == "in": # If in-order, now yield the current
        yield (node.key, node.data) # node
    for childKey, childData in self.__traverse( # Recursively
        node.rightChild, traverseType): # traverse right subtree
        yield (childKey, childData) # yielding its nodes
    if traverseType == "post": # If post-order, yield the current
        yield (node.key, node.data) # node after all the others

```

There is an important but subtle point to note in calling the `__traverse()` method. The public `traverse_rec()` method returns the result of calling the private `__traverse()` method and does not just simply call it as a subroutine. The reason is that the `traverse()` method itself is not the generator; it has no `yield` statements. It must return the iterator produced by the call to `__traverse()`, which will be used by the `traverse_rec()` caller to iterate over the nodes.

Inside the `__traverse()` method, there are a series of `if` statements. The first one tests the base case. If `node` is `None`, then this is an empty tree (or subtree). It returns to indicate the iterator has hit the end (which Python converts to a `StopIteration` exception). The next `if` statement checks whether the traversal type is pre-order, and if it is, it yields the node's key and data. Remember that the iterator will be paused at this point while control passes back to its caller. That is where the node will be "visited." After the processing is done, the

caller's loop invokes this iterator to get the next node. The iterator resumes processing right after the `yield` statement, remembering all the context.

When the iterator resumes (or if the order was something other than pre-order), the next step is a `for` loop. This is a recursive generator to perform the traversal of the left subtree. It calls the `__traverse()` method on the node's `leftChild` using the same `traverseType`. That creates its own iterator to process the nodes in that subtree. As nodes are yielded back as `key, data` pairs, this higher-level iterator yields them back to its caller. This loop construction produces a nested stack of iterators, just like the nested invocations of `i_o_trav()` shown in [Figure 8-12](#). When each iterator returns at the end of its work, it raises a `StopIteration`. The enclosing iterator catches each exception, so the various levels don't interfere with one another.

The rest of the `__traverse()` method is straightforward. After finishing the loop over all the nodes in the left subtree, the next `if` statement checks for the in-order traversal type and yields the node's key and data, if that's the ordering. The node gets processed between the left and right subtrees for an in-order traversal. After that, the right subtree is processed in its own loop, yielding each of the visited nodes back to its caller. After the right subtree is done, a check for post-order traversal determines whether the node should be yielded at this stage or not. After that, the `__traverse()` generator is done, ending its caller's loop.

Making the Generator Efficient

The recursive generator has the advantage of structural simplicity. The base cases and recursive calls follow the node and child structure of the tree. Developing the prototype and proving its correct behavior flow naturally from this structure.

The generator does, however, suffer some inefficiency in execution. Each invocation of the `__traverse()` method invokes two loops: one for the left and one for the right child. Each of those loops creates a new iterator to yield the items from their subtrees back through the iterator created by this invocation of the `__traverse()` method itself. That layering of iterators extends from the root down to each leaf node.

Traversing the N items in the tree should take $O(N)$ time, but creating a stack of iterators from the root down to each leaf adds complexity that's proportional

to the depth of the leaves. The leaves are at $O(\log N)$ depth, in the best case. That means the overall traversal of N items will take $O(N \times \log N)$ time.

To achieve $O(N)$ time, you need to apply the method discussed at the end of [Chapter 6](#) and use a stack to hold the items being processed. The items include both `Node` structures and the (key, data) pairs stored at the nodes to be traversed in a particular order. [Listing 8-7](#) shows the code.

Listing 8-7 *The Nonrecursive `traverse()` Generator*

```
from LinkStack import *

class BinarySearchTree(object): # A binary search tree class
...
    def traverse(self,          # Non-recursive generator to traverse
                  traverseType='in'): # tree in pre, in, or post order
    if traverseType not in [ # Verify traversal type is an
        'pre', 'in', 'post']: # accepted value
        raise ValueError(
            "Unknown traversal type: " + str(traverseType))

    stack = Stack() # Create a stack
    stack.push(self.__root) # Put root node in stack

    while not stack.isEmpty(): # While there is work in the stack
        item = stack.pop() # Get next item
        if isinstance(item, self.__Node): # If it's a tree node
            if traverseType == 'post': # For post-order, put it last
                stack.push((item.key, item.data))
            stack.push(item.rightChild) # Traverse right child
            if traverseType == 'in': # For pre-order, put item 2nd
                stack.push((item.key, item.data))
            stack.push(item.leftChild) # Traverse left child
            if traverseType == 'pre': # For pre-order, put item 1st
                stack.push((item.key, item.data))
        elif item: # Every other non-None item is a
            yield item # (key, value) pair to be yielded
```

The nonrecursive method combines the two parts of the recursive approach into a single `traverse()` method. The same check for the validity of the traversal type happens at the beginning. The next step creates a stack, using the `Stack` class built on a linked list from [Chapter 5](#) (defined in the `LinkStack` module).

Initially, the method pushes the root node of the tree on the stack. That means the remaining work to do is the entire tree starting at the root. The `while` loop that follows works its way through the remaining work until the stack is empty.

At each pass through the `while` loop, the top item of the stack is popped off. Three kinds of items could be on the stack: a `Node` item, a `(key, data)` tuple, or `None`. The latter happens if the tree is empty and when it processes the leaf nodes (and finds their children are `None`).

If the top of the stack is a `Node` item, the `traverse()` method determines how to process the node's data and its children based on the requested traversal order. It pushes items onto the stack to be processed on subsequent passes through the `while` loop. Because the items will be popped off the stack in the reverse order from the way they were pushed onto it, it starts by handling the case for post-order traversal.

In post-order, the first item pushed is the node's `(key, data)` tuple. Because it is pushed first, it will be processed last overall. The next item pushed is the node's right child. In post-order, this is traversed just before processing the node's data. For the other orders, the right child is always the last node processed.

After pushing on the right child, the next `if` statement checks whether the in-order traversal was requested. If so, it pushes the node's `(key, data)` tuple on the stack to be processed in-between the two child nodes. That's followed by pushing the left child on the stack for processing.

Finally, the last `if` statement checks whether the pre-order traversal was requested and then pushes the node's data on the stack for processing before the left and right children. It will be popped off during the next pass through the `while` loop. That completes all the work for a `Node` item.

The final `elif` statement checks for a non-`None` item on the stack, which must be a `(key, data)` tuple. When the loop finds such a tuple, it yields it back to the caller. The `yield` statement ensures that the `traverse()` method becomes a generator, not a function.

The loop doesn't have any explicit handling of the `None` values that get pushed on the stack for empty root and child links. The reason is that there's nothing to do for them: just pop them off the stack and continue on to the remaining work.

Using the stack, you have now made an $O(N)$ generator. Each node of the tree is visited exactly once, pushed on the stack, and later popped off. Its key-data

pairs and child links are also pushed on and popped off exactly once. The ordering of the node visits and child links follows the requested traversal ordering. Using the stack and carefully reversing the items pushed onto it make the code slightly more complex to understand but improve the performance.

Using the Generator for Traversing

The generator approach (both recursive and stack-based) makes the caller's loops easy. For example, if you want to collect all the items in a tree whose data is below the average data value, you could use two loops:

```
total, count = 0, 0
for key, data in random_tree.traverse('pre'):
    total += data
    count += 1
average = total / count
below_average = []
for key, data in random_tree.traverse('in'):
    if data <= average:
        below_average.append((key, data))
```

The first loop counts the number of items in `random_tree` and sums up their data values. The second loop finds all the items whose data is below the average and appends the key and data pair to the `below_average` list. Because the second loop is done in in-order, the keys in `below_average` are in ascending order. Being able to reference the variables that accumulate results—`total`, `count`, and `below_average`—without defining some global (or nonlocal) variables outside a function body, makes using the generator very convenient for traversal.

Traversing with the Visualization Tool

The Binary Search Tree Visualization tool allows you to explore the details of traversal using generators. You can launch any of the three kinds of traversals by selecting the Pre-order Traverse, In-order Traverse, or Post-order Traverse buttons. In each case, the tool executes a simple loop of the form

```
for key, data in tree.traverse("pre"):
    print(key)
```

To see the details, use the Step button (you can launch an operation in step mode by holding down the Shift key when selecting the button). In the code window, you first see the short traversal loop. The example calls the `traverse()` method to visit all the keys and data in a loop using one of the orders such as `pre`.

[Figure 8-14](#) shows a snapshot near the beginning of a pre-order traversal. The code for the `traverse()` method appears at the lower right. To the right of the tree above the code, the `stack` is shown. The nodes containing keys 59 and 94 are on the stack. The top of the stack was already popped off and moved to the top right under the `item` label. It shows the key, 77, with a comma separating it from its colored rectangle to represent the (key, data) tuple that was pushed on the stack. The `yield` statement is highlighted, showing that the `traverse()` iterator is about to yield the key and data back to caller. The loop that called `traverse()` has scrolled off the code display but will be shown on the next step.



Figure 8-14 *Traversing a tree in pre-order using the `traverse()` iterator*

When control returns to the calling loop, the `traverse()` iterator disappears from the code window and so does the stack, as shown in [Figure 8-15](#). The `key` and `data` variables are now bound to 77 and the root node's data. The `print` statement is highlighted because the program is about to print the key in the output box along the bottom of the tree. The next step shows key 77 being copied to the output box.



Figure 8-15 *The loop calling the `traverse()` iterator*

After printing, control returns to the `for key, data in tree.traverse('pre')` loop. That pushes the `traverse()` iterator back on the code display, along with its stack similar to [Figure 8-14](#). The `while` loop in the iterator finds that the stack is not empty, so it pops off the top item. That item is node 59, the left child of node 77. The process repeats by pushing on node 59's children and the node's key, data pair on the stack. On the next loop iteration, that tuple is popped off the stack, and it is yielded back to the print loop.

The processing of iterators is complex to describe, and the visualization tool makes it easier to follow the different levels and steps than reading a written description. Try stepping through the processing of several nodes, including when the iterator reaches a leaf node and pushes `None` on the stack. The stack guides the iterator to return to nodes that remain to be processed.

Traversal Order

What's the point of having three traversal orders? One advantage is that in-order traversal guarantees an ascending order of the keys in binary search trees. There's a separate motivation for pre- and post-order traversals. They are very useful if you're writing programs that *parse* or analyze algebraic expressions. Let's see why that is the case.

A binary tree (not a binary search tree) can be used to represent an algebraic expression that involves binary arithmetic operators such as `+`, `-`, `/`, and `*`. The root node and every nonleaf node hold an operator. The leaf nodes hold either a variable name (like `A`, `B`, or `C`) or a number. Each subtree is a valid algebraic expression.

For example, the binary tree shown in [Figure 8-16](#) represents the algebraic expression

$$(A+B) * C - D / E$$

This is called **infix notation**; it's the notation normally used in algebra. (For more on infix and postfix, see the section “[Parsing Arithmetic Expressions](#)” in [Chapter 4](#).) Traversing the tree in order generates the correct in-order sequence $A+B * C - D / E$, but you need to insert the parentheses yourself to get the expected order of operations. Note that subtrees form their own subexpressions like the $(A+B) * C$ outlined in the figure.

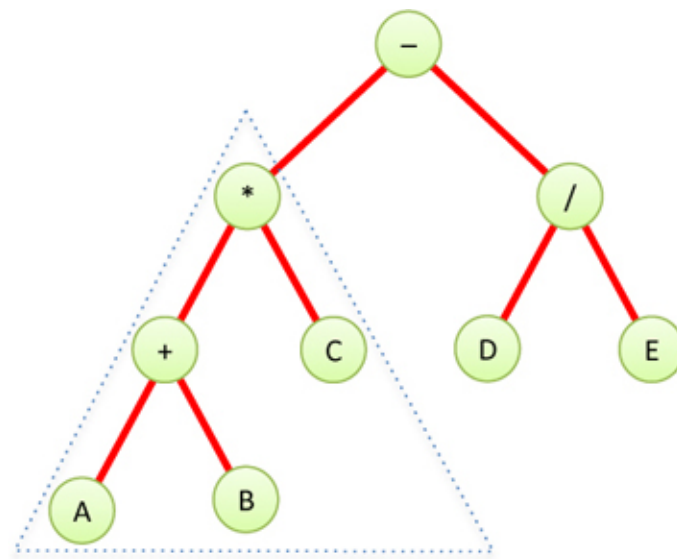


Figure 8-16 *Binary tree representing an algebraic expression*

What does all this have to do with pre-order and post-order traversals? Let's see what's involved in performing a pre-order traversal. The steps are

1. Visit the node.
2. Call itself to traverse the node's left subtree.
3. Call itself to traverse the node's right subtree.

Traversing the tree shown in [Figure 8-16](#) using pre-order and printing the node's value would generate the expression

$$-*+ABC/DE$$

This is called **prefix notation**. It may look strange the first time you encounter it, but one of its nice features is that parentheses are never required; the expression is unambiguous without them. Starting on the left, each operator is applied to the next two things to its right in the expression, called the **operands**. For the first operator, $-$, these two things are a product expression, $*+ABC$, and a division expression, $/DE$. For the second operator, $*$, the two things are a sum expression, $+AB$, and a single variable, C . For the third operator, $+$, the two things it operates on are the variables, A and B , so this last expression would be $A+B$ in in-order notation. Finally, the fourth operator, $/$, operates on the two variables D and E .

The third kind of traversal, post-order, contains the three steps arranged in yet another way:

1. Call itself to traverse the node's left subtree.
2. Call itself to traverse the node's right subtree.
3. Visit the node.

For the tree in [Figure 8-16](#), visiting the nodes with a post-order traversal generates the expression

$$AB+C*DE/-$$

This is called **postfix notation**. It means “apply the last operator in the expression, $-$, to the two things immediately to the left of it.” The first thing is $AB+C*$, and the second thing is $DE/$. Analyzing the first thing, $AB+C*$, shows its meaning to be “apply the $*$ operator to the two things immediately to the left of it, $AB+$ and C .” Analyzing the first thing of that expression, $AB+$, shows its meaning to be “apply the $+$ operator to the two things immediately to the left of it, A and B .” It's hard to see initially, but the “things” are always one of three kinds: a single variable, a single number, or an expression ending in a binary operator.

To process the meaning of a postfix expression, you start from the last character on the right and interpret it as follows. If it's a binary operator, then you repeat the process to interpret two subexpressions on its left, which become the operands of the operator. If it's a letter, then it's a simple variable, and if it's a number, then it's a constant. For both variables and numbers, you “pop” them off the right side of the expression and return them to the process of the enclosing expression.

We don't show the details here, but you can easily construct a tree like that in [Figure 8-16](#) by using a postfix expression as input. The approach is analogous to that of evaluating a postfix expression, which you saw in the `PostfixTranslate.py` program in [Chapter 4](#) and its corresponding InfixCalculator Visualization tool. Instead of storing operands on the stack, however, you store entire subtrees. You read along the postfix string as you did in the `PostfixEvaluate()` method. Here are the steps when you encounter an operand (a variable or a number):

1. Make a tree with one node that holds the operand.
2. Push this tree onto the stack.

Here are the steps when you encounter an operator, O:

1. Pop two operand trees R and L off the stack (the top of the stack has the rightmost operand, R).
2. Create a new tree T with the operator, O, in its root.
3. Attach R as the right child of T.
4. Attach L as the left child of T.
5. Push the resulting tree, T, back on the stack.

When you're done evaluating the postfix string, you pop the one remaining item off the stack. Somewhat amazingly, this item is a complete tree depicting the algebraic expression. You can then see the prefix and infix representations of the original postfix notation (and recover the postfix expression) by traversing the tree in one of the three orderings we described. We leave an implementation of this process as an exercise.

Finding Minimum and Maximum Key Values

Incidentally, you should note how easy it is to find the minimum and maximum key values in a binary search tree. In fact, this process is so easy that we don't include it as an option in the visualization tool. Still, understanding how it works is important.

For the minimum, go to the left child of the root; then go to the left child of that child, and so on, until you come to a node that has no left child. This node is the minimum. Similarly, for the maximum, start at the root and follow the

right child links until they end. That will be the maximum key in the tree, as shown in [Figure 8-17](#).



Figure 8-17 *Minimum and maximum key values of a binary search tree*

Here's some code that returns the minimum node's data and key values:

```
def minNode(self):          # Find and return node with minimum key
    if self.isEmpty():      # If the tree is empty, raise exception
        raise Exception("No minimum node in empty tree")
    node = self.__root      # Start at root
    while node.leftChild:   # While node has a left child,
        node = node.leftChild # follow left child reference
    return (node.data, node.key) # return final node data and key
```

Finding the maximum is similar; just swap the right for the left child. You learn about an important use of finding the minimum value in the next section about deleting nodes.

Deleting a Node

Deleting a node is the most complicated common operation required for binary search trees. The fundamental operation of deletion can't be ignored, however, and studying the details builds character.

You start by verifying the tree isn't empty and then finding the node you want to delete, using the same approach you saw in `__find()` and `insert()`. If the node isn't found, then you're done. When you've found the node and its parent, there are three cases to consider:

1. The node to be deleted is a leaf (has no children).
2. The node to be deleted has one child.
3. The node to be deleted has two children.

Let's look at these three cases in turn. The first is easy; the second, almost as easy; and the third, quite complicated.

Case 1: The Node to Be Deleted Has No Children

To delete a leaf node, you simply change the appropriate child field in the node's parent to `None` instead of to the node. The node object still exists, but it is no longer part of the tree, as shown when deleting node 17 in [Figure 8-18](#).



Figure 8-18 *Deleting a node with no children*

If you're using a language like Python or Java that has garbage collection, the deleted node's memory will eventually be reclaimed for other uses (if you eliminate all references to it in the program). In languages that require explicit allocation and deallocation of memory, the deleted node should be released for reuse.

Using the Visualization Tool to Delete a Node with No Children

Try deleting a leaf node using the Binary Search Tree Visualization tool. You can either type the key of a node in the text entry box or select a leaf with your pointer device and then select Delete. You see the program use `__find()` to locate the node by its key, copy it to a temporary variable, set the parent link to

None, and then “return” the deleted key and data (in the form of its colored background).

Case 2: The Node to Be Deleted Has One Child

This second case isn’t very difficult either. The node has only two edges: one to its parent and one to its only child. You want to “cut” the node out of this sequence by connecting its parent directly to its child. This process involves changing the appropriate reference in the parent (`leftChild` or `rightChild` or `__root`) to point to the deleted node’s child. Figure 8-19 shows the deletion of node 16, which has only one child.

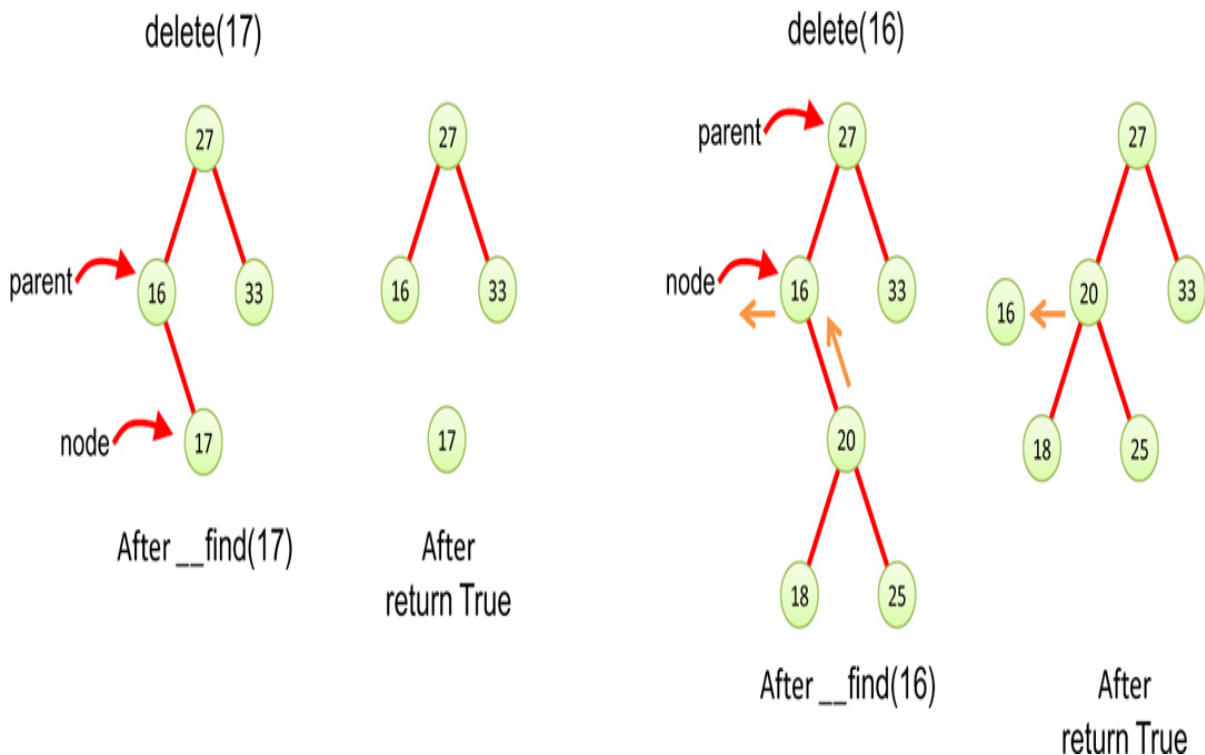


Figure 8-19 *Deleting a node with one child*

After finding the node and its parent, the delete method has to change only one reference. The deleted node, key 16 in the figure, becomes disconnected from the tree (although it may still have a child pointer to the node that was promoted up (key 20). Garbage collectors are sophisticated enough to know that they can reclaim the deleted node without following its links to other nodes that might still be needed.

Now let's go back to the case of deleting a node with no children. In that case, the delete method also made a single change to replace one of the parent's child pointers. That pointer was set to `None` because there was no replacement child node. That's a similar operation to Case 2, so you can treat Case 1 and Case 2 together by saying, "If the node to be deleted, D, has 0 or 1 children, replace the appropriate link in its parent with either the left child of D, if it isn't empty, or the right child of D." If both child links from D are `None`, then you've covered Case 1. If only one of D's child links is non-`None`, then the appropriate child will be selected as the parent's new child, covering Case 2. You promote either the single child or `None` into the parent's child (or possibly `__root`) reference.

Using the Visualization Tool to Delete a Node with One Child

Let's assume you're using the visualization tool on the tree in [Figure 8-5](#) and deleting node 61, which has a right child but no left child. Click node 61 and the key should appear in the text entry area, enabling the Delete button. Selecting the button starts another call to `__find()` that stops with `current` pointing to the node and `parent` pointing to node 59.

After making a copy of node 61, the animation shows the right child link from node 59 being set to node 61's right child, node 62. The original copy of node 61 goes away, and the tree is adjusted to put the subtree rooted at node 62 into its new position. Finally, the copy of node 61 is moved to the output box at the bottom.

Use the visualization tool to generate new trees with single child nodes and see what happens when you delete them. Look for the subtree whose root is the deleted node's child. No matter how complicated this subtree is, it's simply moved up and plugged in as the new child of the deleted node's parent.

Python Code to Delete a Node

Let's now look at the code for at least Cases 1 and 2. [Listing 8-8](#) shows the code for the `delete()` method, which takes one argument, the key of the node to delete. It returns either the data of the node that was deleted or `None`, to indicate the node was not found. That makes it behave somewhat like the methods for popping an item off a stack or deleting an item from a queue. The difference is that the node must be found inside the tree instead of being at a known position in the data structure.

Listing 8-8 The delete() Method of BinarySearchTree

```
class BinarySearchTree(object): # A binary search tree class
...
    def delete(self, goal): # Delete a node whose key matches goal
        node, parent = self.__find(goal) # Find goal and its parent
        if node is not None: # If node was found,
            return self.__delete( # then perform deletion at node
                parent, node) # under the parent

    def __delete(self, # Delete the specified node in the tree
        parent, node): # modifying the parent node/tree
        deleted = node.data # Save the data that's to be deleted
        if node.leftChild: # Determine number of subtrees
            if node.rightChild: # If both subtrees exist,
                self.__promote_successor( # Then promote successor to
                    node) # replace deleted node
            else: # If no right child, move left child up
                if parent is self: # If parent is the whole tree,
                    self.__root = node.leftChild # update root
                elif parent.leftChild is node: # If node is parent's left,
                    parent.leftChild = node.leftChild # child, update left
                else: # else update right child
                    parent.rightChild = node.leftChild
            else: # No left child; so promote right child
                if parent is self: # If parent is the whole tree,
                    self.__root = node.rightChild # update root
                elif parent.leftChild is node: # If node is parent's left
                    parent.leftChild = node.rightChild # child, then update
                else: # left child link else update
                    parent.rightChild = node.rightChild # right child
        return deleted # Return the deleted node's data
```

Just like for insertion, the first step is to find the node to delete and its parent. If that search does not find the `goal` node, then there's nothing to delete from the tree, and `delete()` returns `None`. If the node to delete is found, the node and its parent are passed to the private `__delete()` method to modify the nodes in the tree.

Inside the `__delete()` method, the first step is to store a reference to the node data being deleted. This step enables retrieval of the node's data after the references to it are removed from the tree. The next step checks how many subtrees the node has. That determines what case is being processed. If both a

left and a right child are present, that's Case 3, and it hands off the deletion to another private method, `__promote_successor()`, which we describe a little later.

If there is only a left subtree of the node to delete, then the next thing to look at is its parent node. If the parent is the `BinarySearchTree` object (`self`), then the node to delete must be the root node, so the left child is promoted into the root node slot. If the parent's left child is the node to delete, then the parent's left child link is replaced with the node's left child to remove the node. Otherwise, the parent's right child link is updated to remove the node.

Notice that working with references makes it easy to move an entire subtree. When the parent's reference to the `node` is updated, the child that gets promoted could be a single node or an immense subtree. Only one reference needs to change. Although there may be lots of nodes in the subtree, you don't need to worry about moving them individually. In fact, they "move" only in the sense of being conceptually in different positions relative to the other nodes. As far as the program is concerned, only the parent's reference to the root of the subtree has changed, and the rest of the contents in memory remain the same.

The final `else` clause of the `__delete()` method deals with the case when the `node` has no left child. Whether or not the `node` has a right child, `__delete()` only needs to update the parent's reference to point at the node's right child. That handles both Case 1 and Case 2. It still must determine which field of the parent object gets the reference to the node's right child, just as in the earlier lines when only the left child was present. It puts the `node.rightChild` in either the `__root`, `leftChild`, or `rightChild` field of the parent, accordingly. Finally, it returns the data of the node that was deleted.

Case 3: The Node to Be Deleted Has Two Children

Now the fun begins. If the deleted node has two children, you can't just replace it with one of these children, at least if the child has its own (grand) children. Why not? Examine [Figure 8-20](#) and imagine deleting node 27 and replacing it with its right subtree, whose root is 33. You are promoting the right subtree, but it has its own children. Which left child would node 33 have in its new position, the deleted node's left child, 16, or node 33's left child, 28? And what do you do with the other left child? You can't just throw it away.

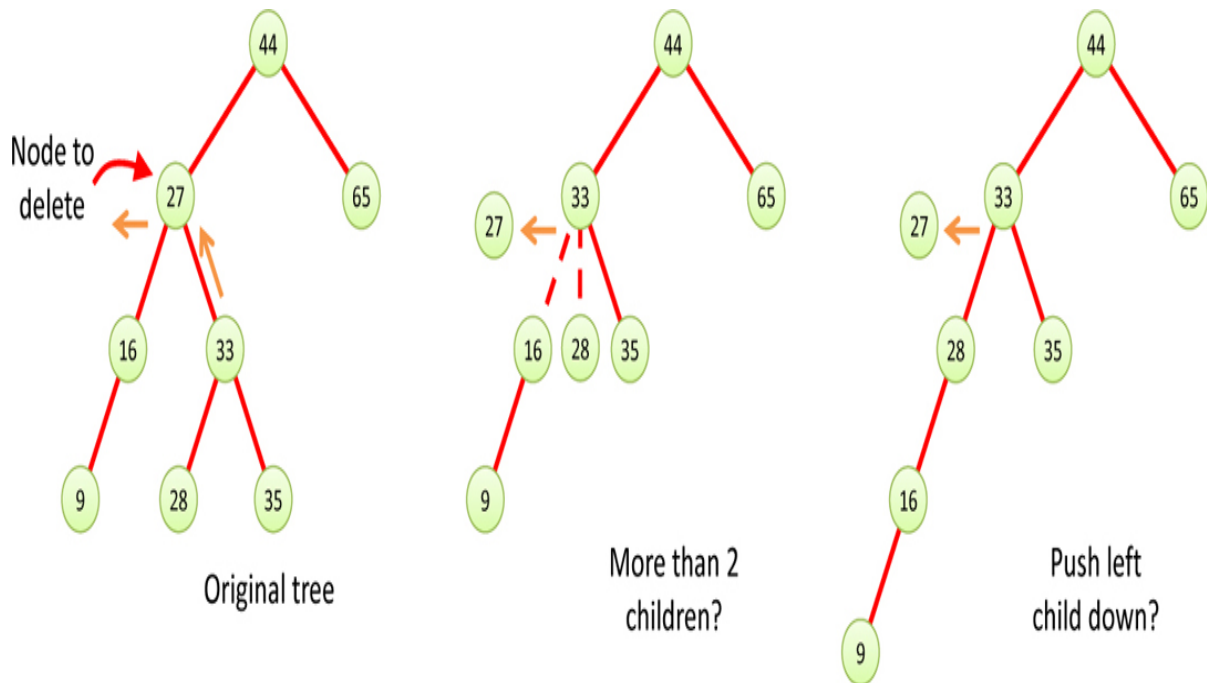


Figure 8-20 Options for deleting a node with two subtrees

The middle option in [Figure 8-20](#) shows potentially allowing three children. That would bring a whole host of other problems because the tree is no longer binary (see [Chapter 9](#) for more on that idea). The right-hand option in the figure shows pushing the deleted node's left child, 16, down and splicing in the new node's left child, 28, above it. That approach looks plausible. The tree is still a binary search tree, at least. The problem, however, is what to do if the promoted node's left child has a complicated subtree of its own (for example, if node 28 in the figure had a whole subtree below it). That could mean following a long path to figure out where to splice the left subtrees together.

We need another approach. The good news is that there's a trick. The bad news is that, even with the trick, there are special cases to consider. Remember that, in a binary search tree, the nodes are arranged in order of ascending keys. For each node, the node with the next-highest key is called its **in-order successor**, or simply its **successor**. In the original tree of [Figure 8-20](#), node 28 is the in-order successor of node 27.

Here's the trick: To delete a node with two children, *replace the node with its in-order successor*. [Figure 8-21](#) shows a deleted node being replaced by its successor. Notice that the nodes are still in order. All it took was a simple replacement. It's going to be a little more complicated if the successor itself has children; we look at that possibility in a moment.

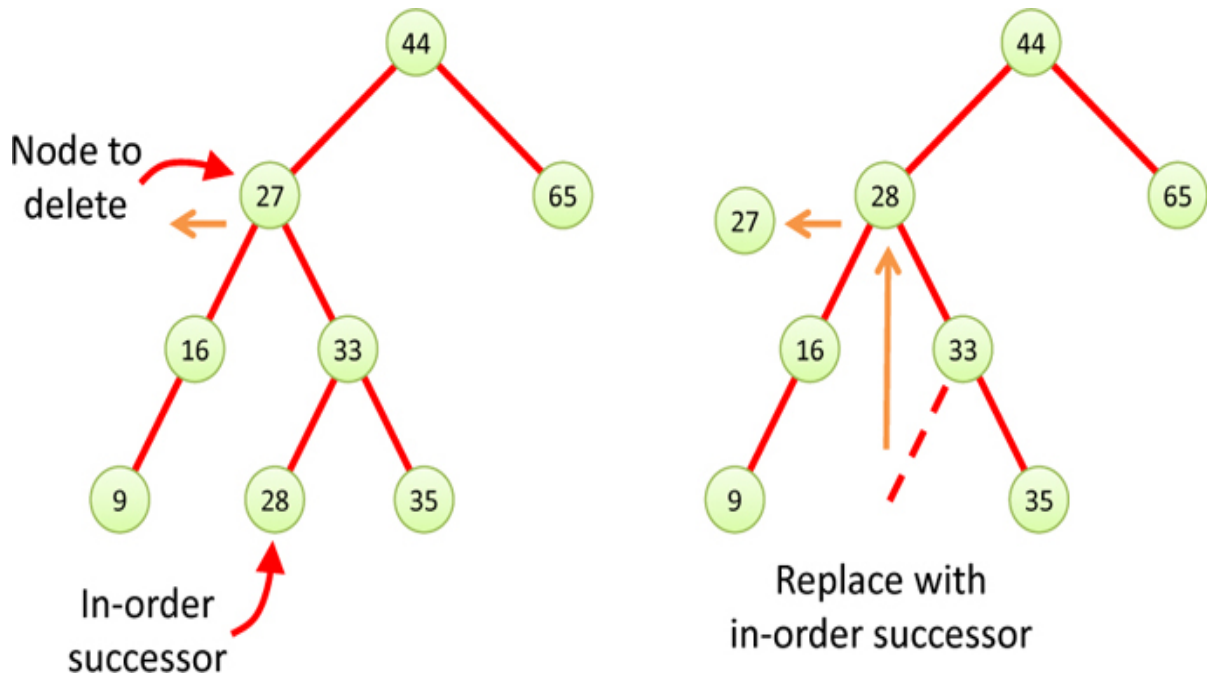


Figure 8-21 *Node replaced by its successor*

Finding the Successor

How do you find the successor of a node? Human beings can do this quickly (for small trees, anyway). Just take a quick glance at the tree and find the next-largest number following the key of the node to be deleted. In [Figure 8-21](#) it doesn't take long to see that the successor of 27 is 28, or that the successor of 35 is 44. The computer, however, can't do things "at a glance"; it needs an algorithm.

Remember finding the node with the minimum or maximum key? In this case you're looking for the *minimum key larger than* the key to be deleted. The node to be deleted has both a left and right subtree because you're working on Case 3. So, you can just look for the minimum key in the right subtree, as illustrated in [Figure 8-22](#). All you need to do is follow the left child links until you find a node with no left child.

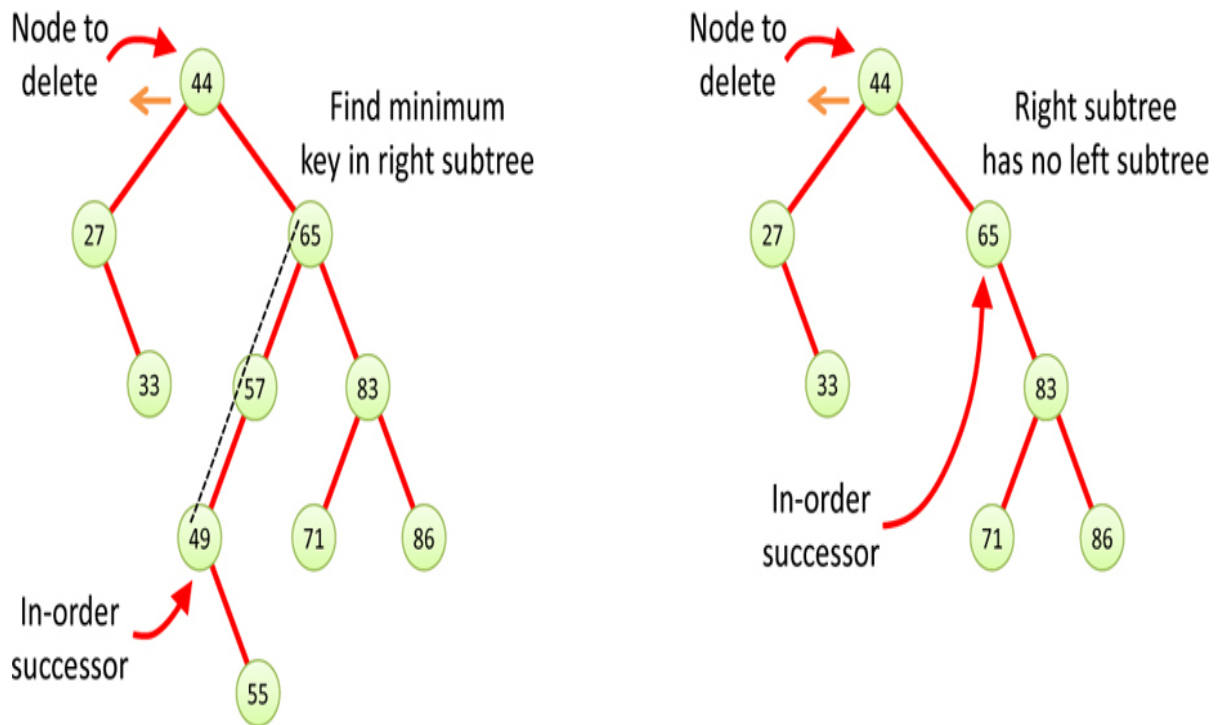


Figure 8-22 *Finding the successor*

What about potential nodes in the trees rooted above the node to be deleted? Couldn't the successor be somewhere in there? Let's think it through. Imagine you seek the successor of node 27 in [Figure 8-22](#). The successor would have to be greater than 27 and less than 33, the key of its right child. Any node with a key between those two values would be inserted somewhere in the left subtree of node 33. Remember that you always search down the binary search tree choosing the path based on the key's relative order to the keys already in the tree. Furthermore, node 33 was placed as the right child of node 27 because it was less than the root node, 44. Any node's right child key must be less than its parent's key if it is the left child of that parent. So going up to parent, grandparent, or beyond (following left child links) only leads to larger keys, and those keys can't be the successor.

There are a couple of other things to note about the successor. If the right child of the original node to delete has no left children, this right child is itself the successor, as shown in the example of [Figure 8-23](#). Because the successor always has an empty left child link, it has at most one child.



Figure 8-23 *The right child is the successor*

Replacing with the Successor

Having found the successor, you can easily copy its key and data values into the node to be deleted, but what do you do with the subtree rooted at the successor node? You can't leave a copy of the successor node in the tree there because the data would be stored in two places, create duplicate keys, and make deleting the successor a problem in the future. So, what's the easiest way to get it out of the tree?

Hopefully, reading [Chapter 6](#) makes the answer jump right out. You can now delete the successor from the tree using a recursive call. You want to do the same operation on the successor that you've been doing on the original node to delete—the one with the `goal` key. What's different is that you only need to do the deletion in a smaller tree, the right subtree where you found the successor. If you tried to do it starting from the root of the tree after replacing the `goal` node, the `__find()` method would follow the same path and end at the node you just replaced. You could get around that problem by delaying the replacement of the key until after deleting the successor, but it's much easier—and more importantly, faster—if you start a new delete operation in the right subtree. There will be much less tree to search, and you can't accidentally end up at the previous `goal` node.

In fact, when you searched for the successor, you followed child links to determine the path, and that gave you both the successor and the successor's parent node. With those two references available, you now have everything needed to call the private `__delete()` method shown in [Listing 8-8](#). You can now define the `__promote_successor()` method, as shown in [Listing 8-9](#).

Remember, this is the method used to handle Case 3—when the node to delete has two children.

Listing 8-9 *The `__promote_successor()` Method of `BinarySearchTree`*

```
class BinarySearchTree(object): # A binary search tree class
...
    def __promote_successor( # When deleting a node with both subtrees,
        self,                # find successor on the right subtree, put
                             # its data in this node, and delete the
                             # successor from the right subtree
        node):
        successor = node.rightChild # Start search for successor in
        parent = node                # right subtree and track its parent
        while successor.leftChild: # Descend left child links until
            parent = successor # no more left links, tracking parent
            successor = successor.leftChild
        node.key = successor.key # Replace node to delete with
        node.data = successor.data # successor's key and data
        self.__delete(parent, successor) # Remove successor node
```

The `__promote_successor()` method takes as its lone parameter the `node` to delete. Because it is going to replace that node's data and key and then delete the successor, it's easier to refer to it as the node to be replaced in this context. To start, it points a `successor` variable at the right child of the `node` to be replaced. Just like the `__find()` method, it tracks the `parent` of the successor node, which is initialized to be the node to be replaced. Then it acts like the `minNode()` method, using a `while` loop to update `successor` with its left child if there is a left child. When the loop exits, `successor` points at the successor node and `parent` to its parent node.

All that's left to do is update the key and data of the node to be replaced and delete the successor node using a recursive call to `__delete()`. Unlike previous recursive methods you've seen, this isn't a call to the same routine where the call occurs. In this case, the `__promote_successor()` method calls `__delete()`, which in turn, could call `__promote_successor()`. This is called **mutual recursion**—where two or more routines call each other.

Your senses should be tingling now. How do you know this mutual recursion will end? Where's the base case that you saw with the “simple” recursion routines? Could you get into an infinite loop of mutually recursive calls? That's a good thing to worry about, but it's not going to happen here. Remember that

deleting a node broke down into three cases. Cases 1 and 2 were for deleting leaf nodes and nodes with one child. Those two cases did not lead to `__promote_successor()` calls, so they are the base cases. When you do call `__promote_successor()` for Case 3, it operates on the subtree rooted at the node to delete, so the only chance that the tree being processed recursively isn't smaller than the original is if the node to delete is the root node. The clincher, however, is that `__promote_successor()` calls `__delete()` only on *successor nodes*—nodes that are guaranteed to have at most one child and at least one level lower in the tree than the node they started on. Those always lead to a base case and never to infinite recursion.

Using the Visualization Tool to Delete a Node with Two Children

Generate a tree with the visualization tool and pick a node with two children. Now mentally figure out which node is its successor, by going to its right child and then following down the line of this right child's left children (if it has any). For your first try, you may want to make sure the successor has no children of its own. On later attempts, try looking at the more complicated situation where entire subtrees of the successor are moved around, rather than a single node.

After you've chosen a node to delete, click the Delete button. You may want to use the Step or Pause/Play buttons to track the individual steps. Each of the methods we've described will appear in the code window, so you can see how it decides the node to delete has two children, locates the successor, copies the successor key and data, and then deletes the successor node.

Is Deletion Necessary?

If you've come this far, you can see that deletion is fairly involved. In fact, it's so complicated that some programmers try to sidestep it altogether. They add a new Boolean field to the `__Node` class, called something like `isDeleted`. To delete a node, they simply set this field to `True`. This is a sort of a “soft” delete, like moving a file to a trash folder without truly deleting it. Then other operations, like `__find()`, check this field to be sure the node isn't marked as deleted before working with it. This way, deleting a node doesn't change the structure of the tree. Of course, it also means that memory can fill up with previously “deleted” nodes.

This approach is a bit of a cop-out, but it may be appropriate where there won't be many deletions in a tree. Be very careful. Assumptions like that tend to come back to haunt you. For example, assuming that deletions might not be frequent for a company's personnel records might encourage a programmer to use the `isDeleted` field. If the company ends up lasting for hundreds of years, there are likely to be more deletions than active employees at some point in the future. The same is true if the company experiences high turnover rates, even over a short time frame. That will significantly affect the performance of the tree operations.

The Efficiency of Binary Search Trees

As you've seen, most operations with trees involve descending the tree from level to level to find a particular node. How long does this operation take? We mentioned earlier that the efficiency of finding a node could range from $O(\log N)$ to $O(N)$, but let's look at the details.

In a full, balanced tree, about half the nodes are on the bottom level. More accurately, in a full, balanced tree, there's exactly one more node on the bottom row than in the rest of the tree. Thus, about half of all searches or insertions or deletions require finding a node on the lowest level. (About a quarter of all search operations require finding the node on the next-to-lowest level, and so on.)

During a search, you need to visit one node on each level. This way, you can get a good idea how long it takes to carry out these operations by knowing how many levels there are. Assuming a full, balanced tree, [Table 8-1](#) shows how many levels are necessary to hold a given number of nodes.

Table 8-1 *Number of Levels for Specified Number of Nodes*

Number of Nodes	Number of Levels
1	1
3	2
7	3
15	4
31	5
...	...
1,023	10
...	...
32,767	15
...	...
1,048,575	20
...	...
33,554,431	25
...	...
1,073,741,823	30

This situation is very much like the ordered array discussed in [Chapter 2](#). In that case, the number of comparisons for a binary search was approximately equal to the base 2 logarithm of the number of cells in the array. Here, if you call the number of nodes in the first column N , and the number of levels in the second column L , you can say that N is 1 less than 2 raised to the power L , or

$$N = 2^L - 1$$

Adding 1 to both sides of the equation, you have

$$N + 1 = 2^L$$

Using what you learned in [Chapter 2](#) about logarithms being the inverse of raising a number to a power, you can take the logarithm of both sides and rearrange the terms to get

$$\log_2(N + 1) = \log_2(2^L) = L$$